



Sorting

- Concept of Algorithm.
- Difference Between Algorithm And Program.
- Quality Of Good Algorithm.
- Greedy Algorithm.
- Divide And Conquer Algorithm.
- Algorithm Complexity.
- Time Complexity.
- Space Complexity.
- Time Complexity Examples.
- Bubble Sort
- Insertion Sort

- Algorithm is a step-by-step procedure, which defines a set of instructions to be executed in a certain order to get the desired output. Algorithms are generally created independent of underlying languages, i.e. an algorithm can be implemented in more than one programming language.
- In computer programming terms, an algorithm is a set of well-defined instructions to solve a particular problem. It takes a set of input and produces a desired output. For example,
- An algorithm to add two numbers:
 1. Take two number inputs
 2. Add numbers using the + operator
 3. Display the result

Algorithm

- Design
- Domain Knowledge
- any Language
- H/W and Os
- Analyse

Program

- Implementation
- Programmer
- Programming Language
- H/W and Os
- Testing

- Input and output should be defined precisely.
- Each step in the algorithm should be clear and unambiguous.
- Algorithms should be most effective among many different ways to solve a problem.
- An algorithm shouldn't include computer code. Instead, the algorithm should be written in such a way that it can be used in different programming languages.

- A greedy algorithm is an approach for solving a problem by selecting the best option available at the moment. It doesn't worry whether the current best result will bring the overall optimal result.
- The algorithm never reverses the earlier decision even if the choice is wrong. It works in a top-down approach.
- This algorithm may not produce the best result for all the problems. It's because it always goes for the local best choice to produce the global best result.

This technique can be divided into the following three parts:

Divide: This involves dividing the problem into some sub problem.

Conquer: Sub problem by calling recursively until sub problem solved.

Combine: The Sub problem Solved so that we will get find problem solution.

- Suppose X is an algorithm and n is the size of input data, the time and space used by the algorithm X are the two main factors, which decide the efficiency of X .
- **Time Factor** – Time is measured by counting the number of key operations such as comparisons in the sorting algorithm.
- **Space Factor** – Space is measured by counting the maximum memory space required by the algorithm.
- The complexity of an algorithm $f(n)$ gives the running time and/or the storage space required by the algorithm in terms of n as the size of input data.

- Time complexity of an algorithm signifies the total time required by the program to run till its completion.
- The time complexity of algorithms is most commonly expressed using the big O notation. It's an asymptotic notation to represent the time complexity.
- Asymptotic Notations are the expressions that are used to represent the complexity of an algorithm.
- Best Case: In which we analyse the performance of an algorithm for the input, for which the algorithm takes less time or space.
- Worst Case: In which we analyse the performance of an algorithm for the input, for which the algorithm takes long time or space.
- Average Case: In which we analyse the performance of an algorithm for the input, for which the algorithm takes time or space that lies between best and worst case.

- Space complexity of an algorithm represents the amount of memory space required by the algorithm in its life cycle. The space required by an algorithm is equal to the sum of the following two components –
- A fixed part that is a space required to store certain data and variables, that are independent of the size of the problem. For example, simple variables and constants used, program size, etc.
- A variable part is a space required by variables, whose size depends on the size of the problem. For example, dynamic memory allocation, recursion stack space, etc.

Time Complexity Examples

Code 1

```
print "Hello "
```

Complexity: $O(1)$

Code 2

```
for ( i=1 i < n ; i ++ )  
    print "Hello "
```

Complexity: $O(n)$

Code 3

```
for ( i =1 ; i <= n ; i ++ )  
{  
    for( j=1; j<=n; j++)  
    {  
        print "Hello "  
    }  
}
```

Complexity: $O(n^2)$

Code 4

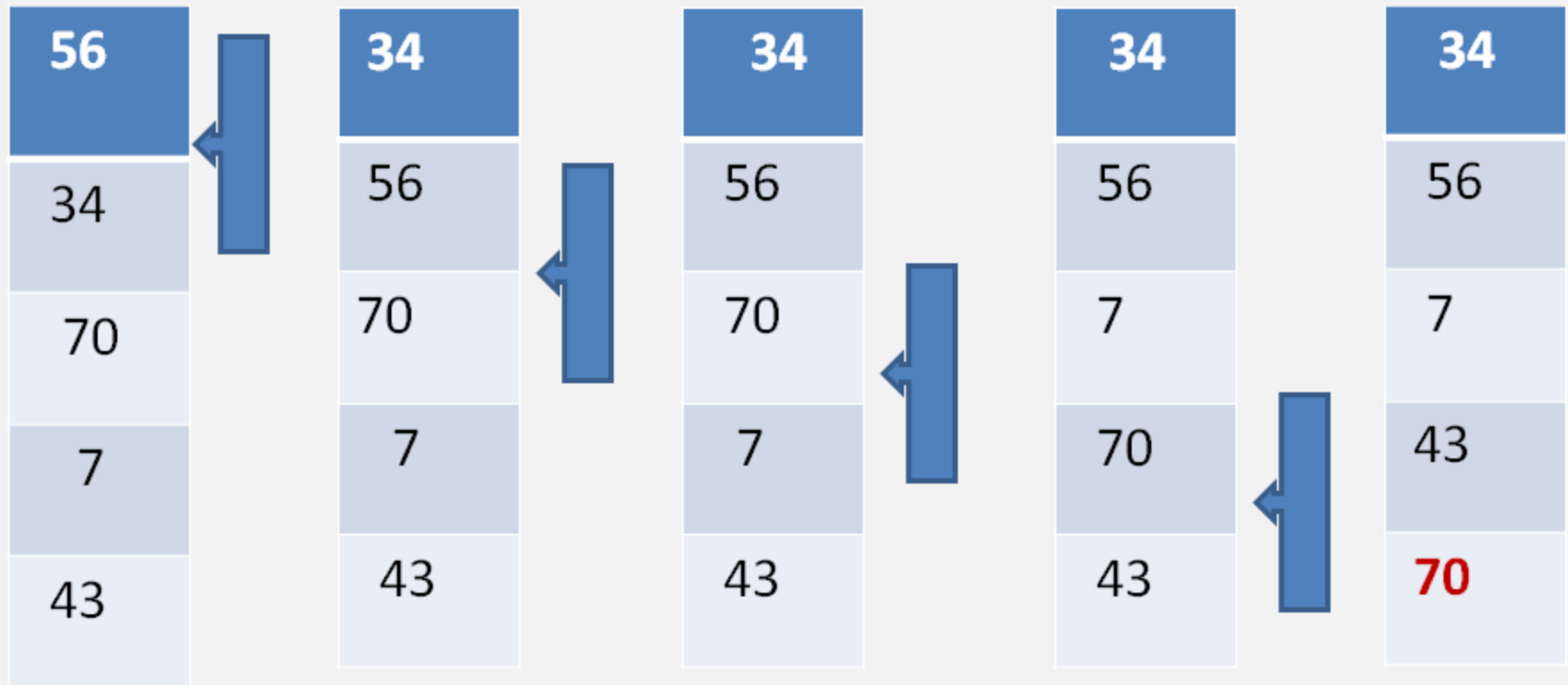
```
for ( i=1 ; i < n ; i = i * 2 )  
{  
    print "Hello "  
}
```

Complexity: $O(\log_2 n)$

Code 5

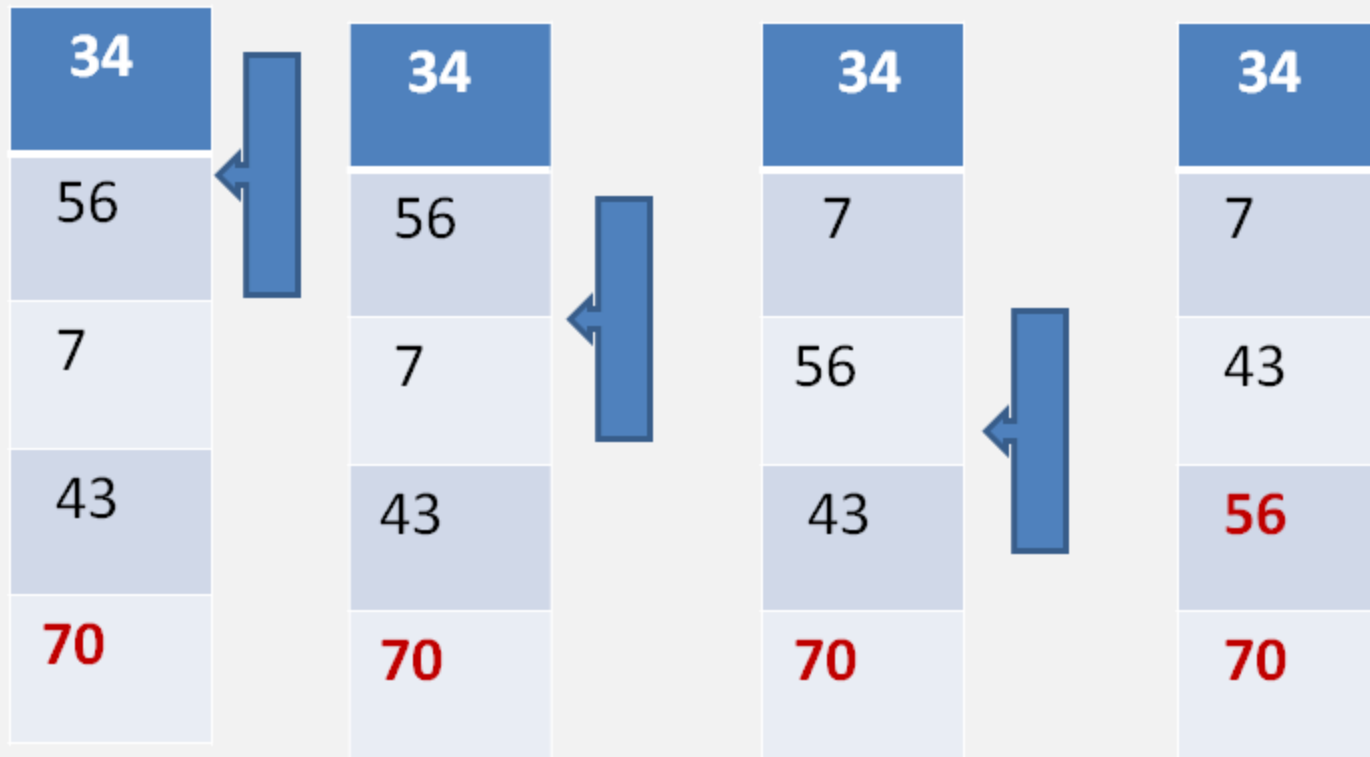
```
        i <=  $\sqrt{n}$   
for ( i=1 ; i2 <= n ; i ++ )  
{  
    print "Hello "  
}
```

Complexity: $O(\sqrt{n})$

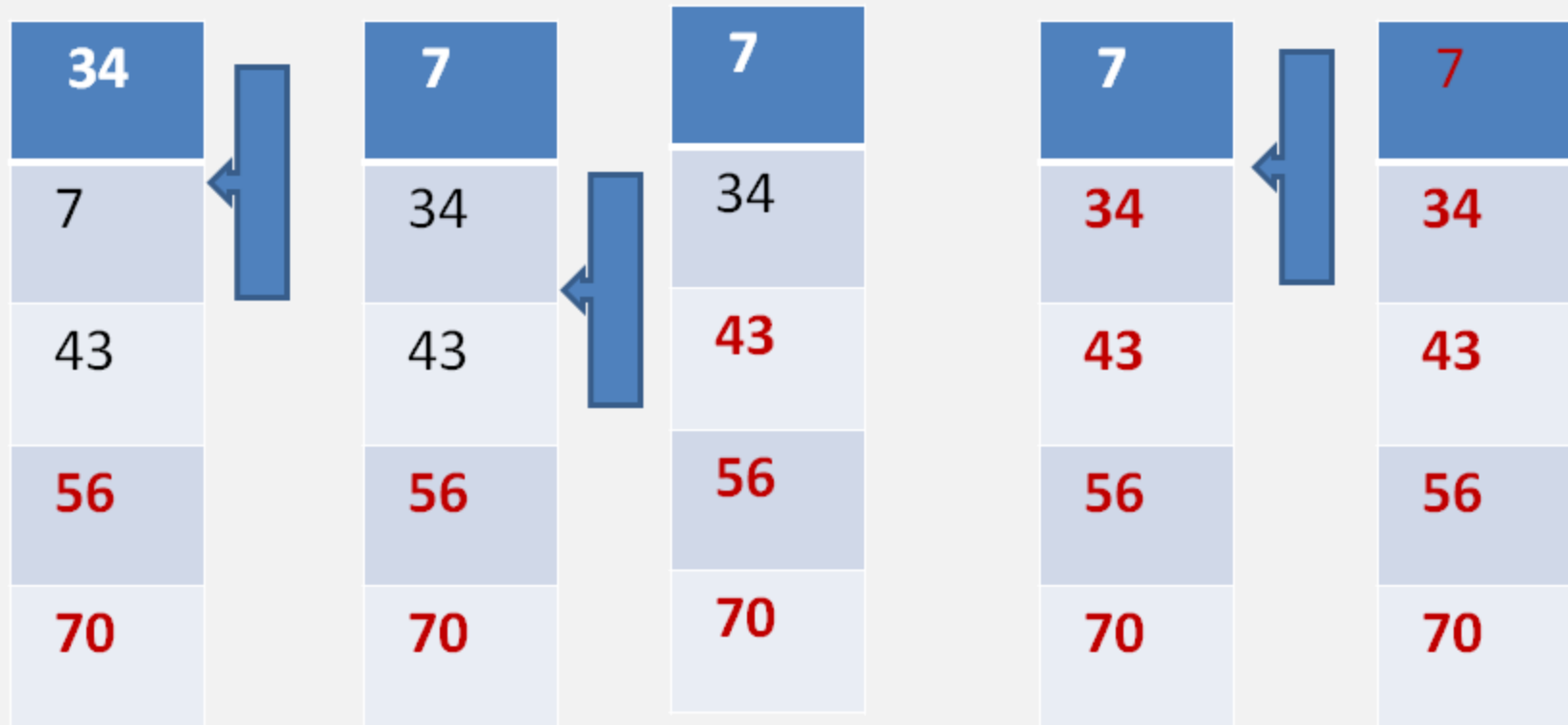


Pass 1 Comparison = 4

Bubble Sort (cont...)



Pass 2 Comparison = 3



Pass 3 Comparison = 2

Pass 4 Comparison = 1

Insertion sort

12	3	25	17	8	11	-4
----	---	----	----	---	----	----

Insertion sort will start from 1 position to it's previous elements and so on .So left subarray is sorted and right sub array is unsorted this way it is checking till end

3	12	25	17	8	11	-4
---	----	----	----	---	----	----

Sorted Array

Unsorted Array

3	12	25	7	8	10	13
---	----	----	---	---	----	----

3	7	12	25	8	10	13
---	---	----	----	---	----	----

Insertion sort

3	7	8	12	25	10	13
---	---	---	----	----	----	----



3	7	8	10	12	25	13
---	---	---	----	----	----	----



3	7	8	10	12	13	25
---	---	---	----	----	----	----

